

FYPals

Phase 1

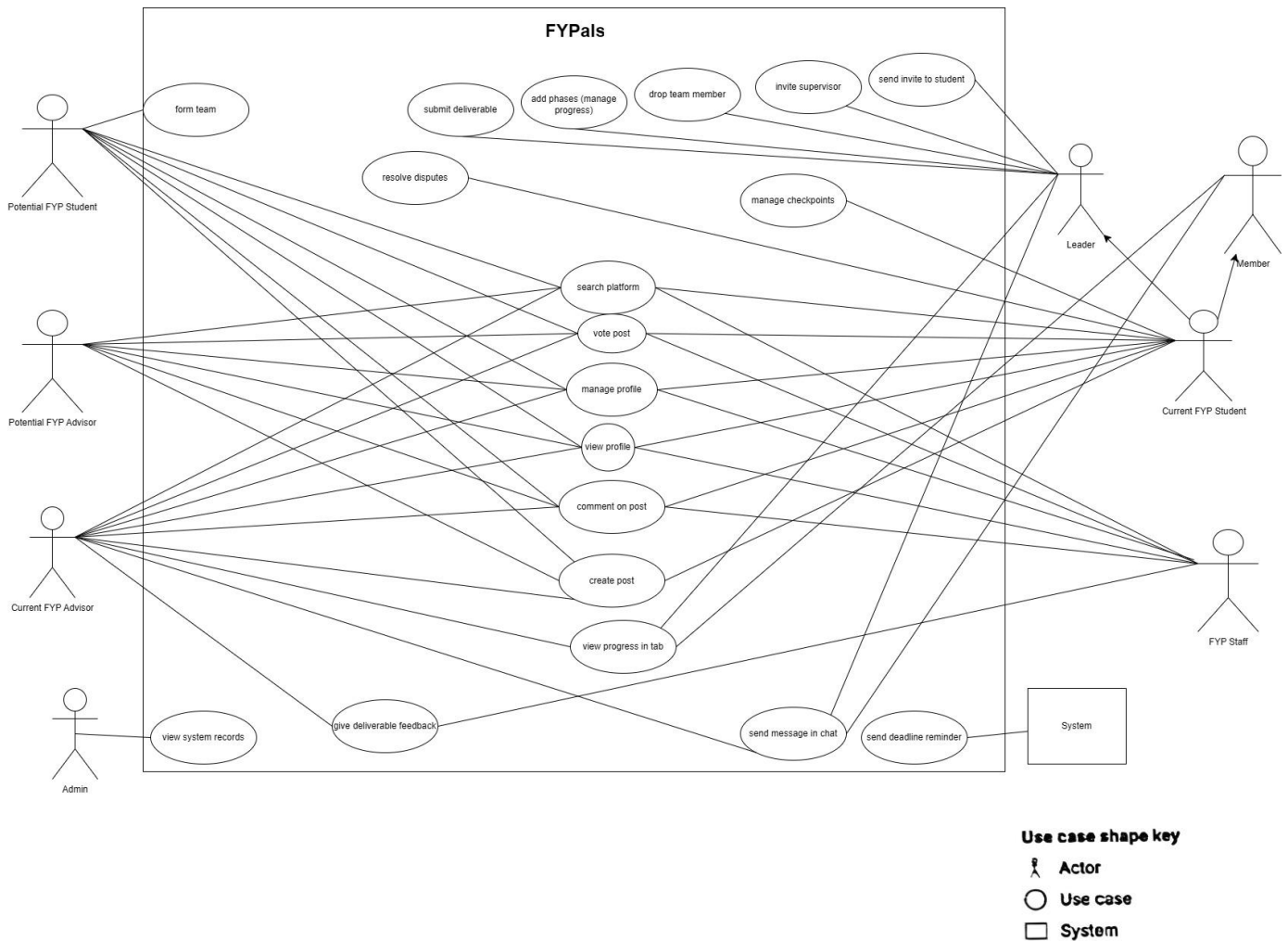
Team 6

Member Name	Member Roll #	Primary Responsibility
Ahmad Ali Khan	24L-3049	UCD, ACD, NFR ,UC-1
Adina Saqib	24L-3003	UCD, ACD, NFR ,UC-8a, UC-8c
Zuhr Faisal	24L-3068	UCD, ACD, NFR ,UC-4, UC-5
Hamna Faisal	24L-2551	UCD, ACD, NFR , UC-6, UC-7
Bilal Kashif	24L-3036	UCD, ACD, NFR ,UC-2 ,UC-9
Manumha Nadeem	24L-3097	UCD, ACD, NFR ,UC-3 , UC-8b

Table of Contents

1. UCD	2
2. Use Cases.....	3
2.1 Form Team.....	3
2.2 Send Invite to Students.....	4
2.3 Send Invite to Advisor	5
2.4 Drop Team Member	6
2.5 Create Post	7
2.6 Vote on Post	8
2.7 Comment on Post	9
2.8 Send Message in Chat	10
2.9 View Progress in Tab	10
2.10 Manage Progress	12
2.11 Manage Checkpoints.....	14
2.12 Search Platform.....	15
2.13 Manage Profile	16
2.14 View Profile	17
2.15 Resolve Dispute	18
2.16 Submit Deliverable.....	20
2.17 Give Deliverable feedback.....	21
2.18 Send Deadline Reminder.....	23
2.19 View System Records	24
3. Non-Functional Requirements	25
3.1 NFR-F1: Performance	25
3.2 NFR-F2: Reliability	25
3.3 NFR-F3: Security	25
3.4 NFR-F4: Usability.....	25
3.5 NFR-F5: Availability	25
3.6 NFR-F6: Maintainability.....	25
4. Analysis Class Diagram (ACD).....	26

1. UCD



2. Use Cases

2.1 Form Team

Identifier	UC-1a	
Name	Form Team	
Summary	Allows a student to create a new FYP team and become its initial leader.	
Priority	High	
Actor(s)	Potential FYP Student	
Pre-condition(s)	Student is logged into the system. Student is not already part of any team. Team formation window is open.	
Post-condition(s)	A new team is created in the system. The student is assigned as Team Leader. The team status is marked as “Incomplete” until minimum members join.	
Typical Course of Action		
S#	Actor Action	System Response
1	Student navigates to the Team Formation page.	
2		System displays options to Create Team or Join Team.
3	Student selects Create Team.	
4		System displays team creation form.
5	Student enters team name and required details.	
6		System validates input fields.
7	Student clicks Create.	
8		System creates the team and assigns the Student as Team Leader.
9		System displays confirmation message and team dashboard.

Alternate Course of Action (Team with same name exists)		
S#	Actor Action	System Response
6		System displays error and asks for a unique team name.
Goes back to step 5.		
Alternate Course of Action 2(Student already in a team)		
S#	Actor Action	System Response
2		System restricts team creation and displays notification.

2.2 Send Invite to Students

Identifier	UC-1b	
Name	Send Invite to Student	
Summary	Allows a Team Leader to invite other Students to join the team.	
Priority	High	
Actor(s)	Current FYP Student Leader	
Pre-condition(s)	Team Leader is logged in. Team exists and has available member slots.	
Post-condition(s)	Invitation is sent to the selected Student. Invited Student receives notification/email.	
Typical Course of Action		
S#	Actor Action	System Response
1	Student(leader) navigates to Team Dashboard.	
2		System displays team details and invite option.
3	Student(leader) selects Invite Member.	
4		System displays search bar for student profiles.
5	Student(leader) searches and selects a Student.	
6		System displays selected student profile.
7	Student(leader) clicks Send Invite.	

8		System sends invitation notification to the Student.
Alternate Course of Action (Student is already in a team)		
S#	Actor Action	System Response
6		System displays error and prevents invitation
Alternate Course of Action 2(Team is already full)		
S#	Actor Action	System Response
8		System prevents invitation and displays message.

2.3 Send Invite to Advisor

Identifier	UC-1c	
Name	Invite Advisor	
Summary	Allows a Team Leader to invite a Potential FYP Advisor to supervise the team.	
Priority	High	
Actor(s)	Current FYP Student Leader	
Pre-condition(s)	Team is formed. Advisor registration exists in the system.	
Post-condition(s)	Invitation request is sent to the Advisor. Advisor receives notification.	
Typical Course of Action		
S#	Actor Action	System Response
1	Leader navigates to Advisor section.	
2		System displays list/search of available Advisors.
3	Leader selects an Advisor profile.	
4		System displays advisor details.
5	Leader clicks Invite Advisor.	
6		System sends supervision request notification.
Alternate Course of Action (Advisor has reached supervision limit)		
S#	Actor Action	System Response
6		System displays error and prevents request.

2.4 Drop Team Member

Identifier	UC-1d	
Name	Drop Team Member	
Summary	This use case allows the Team Leader to remove a member from the team before the team is finalized or approved.	
Priority	High	
Actor(s)	Current FYP Student Leader	
Pre-condition(s)	The system is accessible. The Team Leader is logged in. The team exists and contains more than one member. The team is not locked or finalized by the Advisor/FYP Office.	
Post-condition(s)	The selected member is removed from the team. The team member list is updated. The removed member is notified of the removal.	
Typical Course of Action		
S#	Actor Action	System Response
1	Leader navigates to the Team Management/Dashboard page.	
2		System displays team details and list of members.
3	Leader selects the Drop option next to a team member’s name.	
4		System prompts for confirmation before removal.
5	Leader confirms the removal action.	
6		System validates that the actor is the Team Leader.
7		System removes the selected member from the team.
8		System updates the team member list and notifies the removed member.
Alternate Course of Action (Leader attempts to remove themselves)		
S#	Actor Action	System Response
6		System displays error message stating that the Team Leader cannot remove themselves.

Alternate Course of Action (Team is locked/Finalized)		
S#	Actor Action	System Response
4		System disables the Drop option and displays message that modifications are not allowed

2.5 Create Post

Identifier	UC-2a	
Name	Create Post	
Summary	This use case allows a student to create and publish a new post in the system. A post may represent a project idea, collaboration requirement, or discussion topic related to FYP.	
Priority	High	
Actor(s)	Current FYP Student Leader, Current FYP Student Member, Current FYP Advisor, Potential FYP Student, Potential FYP Advisor	
Pre-condition(s)	User must be authenticated (logged into the system).User must have a completed profile.	
Post-condition(s)	The post is successfully stored in the database. The post becomes visible to other authorized users. The post is available for search, filtering, and feedback.	
Typical Course of Action		
S#	Actor Action	System Response
1	User selects “Create Post” option	
2		System displays post creation interface
3	User enters post title and description	
4		System validates input fields
5	User selects post category (Idea / Project / Requirement)	

6		System records category selection
7	User clicks "Publish"	
8		System stores post in database
9		System confirms successful publication and displays the post
Alternate Course of Action 1 (Missing Fields Required)		
S#	Actor Action	System Response
2		System identifies required fields are empty
3	User corrects input	
Return to step 4		
Alternate Course of Action 2 (System Failure During Publish)		
4		System fails to store post
5		Display error
Return to step 2		

2.6 Vote on Post

Identifier	UC-2b	
Name	Vote Post	
Summary	Allows users to upvote a post.	
Priority	Low	
Actor(s)	Current FYP Student Leader, Current FYP Student Member, Current FYP Advisor, Potential FYP Student, Potential FYP Advisor, FYP Staff	
Pre-condition(s)	User is logged in. A post exists in a forum the user can access.	
Post-condition(s)	Post's vote count increases/decreases by one.	
Typical Course of Action		
S#	Actor Action	System Response
1	User clicks Vote button on a post.	
2		System increments the Vote count.
Alternate Course of Action (User Already Upvoted)		

S#	Actor Action	System Response
1	User already upvoted the post.	
2		System removes the upvote (toggle).
Alternate Course of Action 2 (User Already Upvoted)		
S#	Actor Action	System Response
1	User already downvoted the post.	
2		System removes the downvote (toggle).

2.7 Comment on Post

Identifier	UC-2c	
Name	Comment on Post	
Summary	Allows Students to add comments to posts.	
Priority	Medium	
Actor(s)	Current FYP Student Leader, Current FYP Student Member, Current FYP Advisor, Potential FYP Student, Potential FYP Advisor, FYP Staff	
Pre-condition(s)	User is logged in. A post exists in a forum the user can access.	
Post-condition(s)	Comment is saved and displayed under the post. Post author is notified.	
Typical Course of Action		
S#	Actor Action	System Response
1	User types a comment and clicks Submit.	
2		System saves and displays the comment.
Alternate Course of Action (Empty Comment)		
S#	Actor Action	System Response
1	User tries to submit an empty comment.	
2		System prompts user to enter text.

2.8 Send Message in Chat

Identifier	UC-3	
Name	Send Message in Chat	
Summary	Allows users to send messages in the project chat.	
Priority	High	
Actor(s)	Current FYP Student Leader, Current FYP Student Member, Current FYP Advisor	
Pre-condition(s)	User is logged in. User belongs to a project team with a chat channel.	
Post-condition(s)	The message is delivered to all participants in the chat channel. The message is saved in the chat history.	
Typical Course of Action		
S#	Actor Action	System Response
1	User types a message and clicks Send.	
2		System delivers message to all participants.
Alternate Course of Action (Sending Empty Message)		
S#	Actor Action	System Response
1	User attempts to send an empty message.	
2		System disables the Send button or displays a prompt to enter text.

2.9 View Progress in Tab

Identifier	UC-4a
Name	View Progress in Tab

Summary	Allows Student Members and Advisors to view the real-time status of project phases and checkpoints through a dedicated progress tab.	
Priority	Medium	
Actor(s)	Current FYP Student Leader, Current FYP Student Member, Current FYP Advisor	
Pre-condition(s)	User is logged into the system. User is assigned to or advising the specific project.	
Post-condition(s)	Project progress data is successfully displayed	
Typical Course of Action		
S#	Actor Action	System Response
1	User clicks on the "Progress" tab in the project dashboard.	
2		System fetches the current status of all phases and checkpoints for that project.
3		System calculates the overall completion percentage.
4		System displays a visual progress bar and a detailed list of tasks.
5	User clicks on a specific phase to expand it.	
6		System displays all checkpoints within that phase and their individual statuses.
7	User applies a filter to see only "Completed" tasks.	
8		System updates the view to show only tasks matching the filter.
9	User closes the progress tab.	
10		System returns the user to the main dashboard.
Alternate Course of Action 1 (Advisor Viewing Progress)		
S#	Actor Action	System Response
1	Advisor selects a project from their "Advised Projects" list and clicks the "Progress" tab.	
2		System displays the progress overview along with an additional "Advisor Feedback"

		section.
3	Advisor clicks on the "Review" button next to a completed phase.	
Goes back to Step 4		
Alternate Course of Action 2 (No Data Available)		
S#	Actor Action	System Response
2		System detects that no phases or checkpoints have been created for this project yet.
3		System displays a message: "No progress data available. Please add phases to begin."
11	User exits the tab.	
12		System returns the user to the main menu and ends the process.

2.10 Manage Progress

Identifier	UC-4b	
Name	Manage Progress	
Summary	Allows the Student (Team Leader) to organize and track project progress by creating project phases and defining checkpoints within each phase.	
Priority	High	
Actor(s)	Current FYP Student Leader	
Pre-condition(s)	Leader is logged into the system. Project team has been officially registered.	
Post-condition(s)	New phases and/or checkpoints are added to the project roadmap. Phases and checkpoints are visible to team members and the advisor. The project roadmap is updated accordingly.	
Typical Course of Action		
S#	Actor Action	System Response
1	Leader navigates to Project Roadmap .	
2		System displays the roadmap timeline with options to Add Phase and Add Checkpoint .
3	Leader chooses to Add Phase or Add Checkpoint .	

4		System displays the appropriate form based on the selection.
5	Leader enters the required details (phase information or checkpoint details).	
6	Leader submits the form.	
7		System validates the input data.
8		System saves the phase or checkpoint and updates the roadmap.
9		System displays a success message.
10	Leader views the updated roadmap or checkpoint list.	
Alternate Course of Action 1 (Overlapping Phase Dates)		
S#	Actor Action	System Response
7		System detects that dates overlap with an existing phase.
8		System displays warning: "Dates overlap with existing phase."
9	Leader adjusts the dates.	
Goes back to Step 6		
Alternate Course of Action 2 (Maximum Phases Reached)		
S#	Actor Action	System Response
3	Leader clicks "Add New Phase".	
4		System detects project has reached maximum phase limit.
5		System displays error: "Maximum phase limit reached."
Alternate Course of Action 3 (Missing Required Fields)		
S#	Actor Action	System Response
7		System detects missing required fields (e.g., checkpoint title).
8		System highlights the missing fields and displays an error message.
Alternate Course of Action 4 (Invalid Checkpoint Deadline)		
S#	Actor Action	System Response
7		System detects that the deadline is in the past or beyond the phase end date.
8		System displays a date validation error.
9	Leader enters a valid deadline date.	
Goes back to Step 6		

2.11 Manage Checkpoints

Identifier		UC-4c
Name		Manage Checkpoints
Summary		Allows the Student Leader and Student Members to manage project checkpoints by assigning tasks to team members and updating checkpoint status to track project progress.
Priority		High
Actor(s)		Current FYP Student Leader, Current FYP Student Member
Pre-condition(s)		User is logged into the system. User is part of the project team. Project phases and checkpoints have already been created.
Post-condition(s)		Checkpoints may be assigned to team members. Checkpoint status is updated in the system. Project progress percentage is recalculated. Updated checkpoint information is visible to the team and advisor.
Typical Course of Action		
S#	Actor Action	System Response
1	User navigates to the Project Tasks / Progress tab.	
2		System displays all project checkpoints and team members.
3	Leader selects a checkpoint and chooses Assign Task OR Update Status .	
4		System displays assignment options or status options depending on the selection.
5	Leader selects a team member to assign the checkpoint.	
6	Leader confirms the assignment.	
7		System validates the assignment and records it in the database.
8		System updates the task list and displays a success message.
9	Student selects a checkpoint assigned to them.	
10	Student clicks Change Status .	
11		System displays available status options such as In Progress and In Review .

12	Student selects a status and confirms.	
13		System verifies the role and updates the checkpoint status.
14		System recalculates project progress and displays a success message.
Alternate Course of Action 2 (Unauthorized Status Change)		
S#	Actor Action	System Response
12	Member attempts to select Completed status.	
13		System validates the role and denies the action.
14		System displays an error message indicating insufficient permissions.
Alternate Course of Action 3 (Member Already Overloaded)		
S#	Actor Action	System Response
5	Leader selects a member who already has maximum tasks.	
6		System displays a workload warning message.
7	Leader chooses another member or proceeds anyway.	
Alternate Course of Action 4 (Checkpoint Already Completed)		
S#	Actor Action	System Response
3	Leader selects a checkpoint already marked Completed .	
4		System disables the Assign Task Finished status.

2.12 Search Platform

Identifier	UC-5
Name	Search Platform
Summary	Allows users to search for projects/posts using keywords.
Priority	Medium
Actor(s)	Current FYP Student Leader, Current FYP Student Member, Current FYP Advisor, Potential FYP Student, Potential FYP Advisor, FYP Staff
Pre-condition(s)	User is logged in. Projects exist in the system.
Post-condition(s)	System displays projects matching the search.
Typical Course of Action	

S#	Actor Action	System Response
1	Actor navigates to the search page.	
2		System displays a search bar with options.
3	User enters keywords in search bar.	
4		System displays matching projects/profile.
5	User clicks on a project/profile to view details.	
6		System shows projects/profile information.
Alternate Course of Action (No Project Matches)		
S#	Actor Action	System Response
2	No projects/profile matches the search	
3		System displays "No results found" and suggests refining the search.

2.13 Manage Profile

Identifier	UC-6a	
Name	Manage Profile	
Summary	Allows a Student, Advisor, or FYP Office Staff to create a new profile or update their existing profile information on the platform.	
Priority	High	
Actor(s)	Current FYP Student Leader, Current FYP Student Member, Current FYP Advisor, Potential FYP Student, Potential FYP Advisor, FYP Staff	
Pre-condition(s)	Actor is registered and logged in.	
Post-condition(s)	A new profile is created or an existing profile is updated. Profile information is stored in the system. Updated profile information is visible to other users on the platform.	
Typical Course of Action		
S#	Actor Action	System Response
1	User navigates to the Profile section.	
2		System checks whether a profile already exists.
3		If no profile exists, system

		displays an empty profile creation form .
4		If a profile exists, system displays the current profile information .
5	User enters or modifies profile details (name, bio, skills, contact information).	
6	User clicks Save Profile .	
7		System validates the entered information.
8		System saves the profile information in the database.
9		System displays a confirmation message and shows the updated profile page.
Alternate Course of Action 1 (Missing Required Fields)		
S#	Actor Action	System Response
7		System detects missing required fields.
8		System highlights the missing fields and prompts the user to complete them before saving.
Alternate Course of Action 2 (Invalid Input)		
S#	Actor Action	System Response
7		System detects invalid input values.
8		System displays validation errors and requests corrections before saving.

2.14 View Profile

Identifier	UC-6b
Name	View Profile
Summary	Allows a Student, Advisor, or FYP Office Staff to view a user's profile on the platform.
Priority	Medium
Actor(s)	Current FYP Student Leader, Current FYP Student Member, Current FYP Advisor, Potential FYP Student, Potential FYP Advisor, FYP Staff
Pre-condition(s)	Actor is logged in. The profile being viewed exists in the system.
Post-condition(s)	The profile information is displayed to the actor.
Typical Course of Action	

S#	Actor Action	System Response
1	User searches for or navigates to a user's profile.	
2		System retrieves and displays the profile including name, bio, skills, and contact info.
Alternate Course of Action 1 (Profile Not Found)		
S#	Actor Action	System Response
2		System displays a message indicating the profile does not exist or is not available.

2.15 Resolve Dispute

Identifier	UC-7	
Name	Resolve Disputes	
Summary	Allows students to raise disputes regarding team decisions, tasks, or grades. The Team Leader can resolve the dispute by either rejecting it or accepting it and creating a poll for team members to vote on.	
Priority	High	
Actor(s)	Current FYP Student Leader, Current FYP Student Member	
Pre-condition(s)	Student is logged into the system. Student belongs to a team. A disputable item (task, grade, or decision) exists. For resolution actions, the actor must be the Team Leader .	
Post-condition(s)	The dispute is recorded in the system. The dispute is either rejected or resolved through a team poll . Team members are notified about dispute status updates. If a poll is created, votes are recorded and results are updated.	
Typical Course of Action		
S#	Actor Action	System Response
1	Student navigates to the dispute section.	
2		System displays a list of items eligible for dispute.
3	Student selects an item and clicks Request Dispute .	
4		System displays a form to enter the dispute reason.
5	Student enters the reason and submits the dispute request.	
6		System records the dispute and

		notifies the Team Leader.
7	Team Leader opens the dispute management section.	
8		System displays pending dispute requests.
9	Team Leader reviews the dispute and selects a resolution option.	
10		System provides options to Reject Dispute or Accept and Create Poll .
11	Leader selects Reject Dispute OR Accept and Create Poll .	
12		If rejected, the system records the rejection and notifies the student. If accepted, the system displays a poll creation form.
13	Leader enters poll details and submits.	
14		System creates the poll and notifies team members.
15	Team members view the poll and cast their votes.	
16		System records votes and updates poll results in real time.
Alternate Course of Action 1 (Missing Reason)		
S#	Actor Action	System Response
5		System displays a validation error asking the student to provide a reason before submitting.
Alternate Course of Action 2 (Reject Dispute)		
S#	Actor Action	System Response
11	Leader selects Reject Dispute .	
12		System prompts the leader to enter a rejection reason.
13	Leader enters the reason and confirms.	
14		System marks the dispute as Rejected and notifies the student who raised it.
Alternate Course of Action 3 (Missing Poll Details)		
S#	Actor Action	System Response
13		System highlights missing poll fields and prompts the leader to complete them before submission.
Alternate Course of Action 4 (Already Voted)		
S#	Actor Action	System Response

15	Student attempts to vote again.	
16		System displays a message indicating the student has already voted and cannot vote again.
Alternate Course of Action 5 (Poll Deadline Passed)		
S#	Actor Action	System Response
15	Student attempts to vote after the poll deadline.	
16		System displays a message that the poll has closed and voting is no longer allowed.

2.16 Submit Deliverable

Identifier	UC-8a	
Name	Submit Deliverable	
Summary	Allows a Student to submit a project deliverable through the platform before the deadline.	
Priority	High	
Actor(s)	Current FYP Student Leader	
Pre-condition(s)	Student is logged in and belongs to an assigned team. A deliverable deadline has been set by the Advisor or FYP Office Staff. The submission window is currently open.	
Post-condition(s)	The deliverable is successfully uploaded and saved in the system. The Advisor and FYP Office Staff are notified of the new submission. The submission is marked as Submitted with a timestamp.	
Typical Course of Action		
S#	Actor Action	System Response
1	Leader navigates to the deliverable submission page.	
2		System displays the list of pending deliverables with their deadlines.
3	Leader selects the relevant deliverable.	
4		System displays the submission form with a file upload option.
5	Leader uploads the deliverable file and clicks Submit.	
6		System validates the file format

		and size.
7		System saves the submission and records the timestamp.
8		System marks the deliverable as Submitted and notifies the Advisor and FYP Office Staff.
Alternate Course of Action 1 (Unsupported File Format)		
S#	Actor Action	System Response
6		System displays an error and prompts the user to upload a valid file type.
Alternate Course of Action 2 (Deadline Passed)		
S#	Actor Action	System Response
4		System rejects the submission and displays a message that the deadline has passed.

2.17 Give Deliverable feedback

Identifier	UC-8b	
Name	Give Deliverable Feedback	
Summary	Allows an Advisor or FYP Office Staff to review a submitted deliverable and provide written feedback. Only the Advisor can approve or request changes; Staff can only leave comments.	
Priority	High	
Actor(s)	Current FYP Advisor, FYP Staff	
Pre-condition(s)	User is logged in with an Advisor or FYP Office Staff role. A team has submitted a deliverable.	
Post-condition(s)	Feedback is saved and linked to the deliverable. The team is notified of the feedback. If Advisor approves, the deliverable status is updated to Approved. If Advisor requests changes, the status is updated to Changes Requested and the team can resubmit.	
Typical Course of Action		
S#	Actor Action	System Response
1	Advisor navigates to the submissions page.	
2		System displays the list of submitted deliverables pending

		review.
3	Advisor selects a deliverable to review.	
4		System displays the submitted file along with a feedback form.
5	Advisor writes feedback in the comment box.	
6		System allows text input for the comment.
7	Advisor selects Approve or Request Changes and submits.	
8		System saves the feedback and updates the deliverable status accordingly.
8		System notifies the team of the feedback and the updated status.
Alternate Course of Action 1 (Staff Role - Comment Only)		
S#	Actor Action	System Response
1	FYP Office Staff navigates to the submissions page.	
2		System displays submitted deliverables. Approve and Request Changes options are disabled for Staff role.
3	FYP Office Staff writes a comment and submits.	
4		System saves the comment and notifies the team. No status change is made.
Alternate Course of Action 2 (No Decision Selected)		
S#	Actor Action	System Response
8		System prompts the Advisor to select a decision before submitting.
Alternate Course of Action 3 (Empty Feedback)		
S#	Actor Action	System Response
8		System displays a validation error asking the user to enter feedback before submitting.

2.18 Send Deadline Reminder

Identifier	UC-8c	
Name	Send Deadline Reminder	
Summary	The system automatically sends a reminder notification to Students and Advisors when a deliverable deadline is approaching.	
Priority	High	
Actor(s)	System	
Pre-condition(s)	A deliverable deadline has been set by the Advisor or FYP Office Staff. The deadline has not yet passed. The reminder threshold time has been reached (e.g. 24 hours before deadline).	
Post-condition(s)	A reminder notification is sent to all relevant Students and the Advisor. The notification is logged in the system.	
Typical Course of Action		
S#	Actor Action	System Response
1		System checks all upcoming deliverable deadlines.
2		System identifies deliverables whose deadline is within the reminder threshold.
3		System retrieves the list of Students and Advisors associated with each deliverable.
4		System sends a reminder notification to all relevant Students and the Advisor.
5		System logs the reminder as sent.
Alternate Course of Action 1 (No Upcoming Deadlines)		
S#	Actor Action	System Response
2		System finds no deadlines within the reminder threshold and no notifications are sent.
Alternate Course of Action 2 (Reminder Already Sent)		
S#	Actor Action	System Response
2		System detects a reminder has already been sent for this deadline and skips sending a duplicate notification.

2.19 View System Records

Identifier	UC-9	
Name	View System Records	
Summary	Allows the Admin to access and view all records in the system including users, posts, teams, projects, checkpoints, and submissions.	
Priority	High	
Actor(s)	Admin	
Pre-condition(s)	1. Admin is authenticated. 2. Admin has accessed the Admin Dashboard.	
Post-condition(s)	1. Requested records are displayed. 2. Admin can perform further management actions.	
Typical Course of Action		
S#	Actor Action	System Response
1	Admin logs into Admin Panel	
2		System displays Admin Dashboard
3	Admin selects a database category (Users / Posts / Projects / Teams)	
4		System retrieves relevant records
5		System displays records in tabular searchable format
6	Admin selects a specific record	
7		System displays detailed information
Alternate Course of Action 1 (No Records Found)		
S#	Actor Action	System Response
4		System finds no records
5		System displays "No records available" message
Return to Step 3		

3. Non-Functional Requirements

3.1 NFR-F1: Performance

- 3.1.1 The system SHALL deliver team chat messages within 2 seconds of being sent.
- 3.1.2 The system SHALL return search and filter results within 2 seconds.
- 3.1.3 The system SHALL generate student/project match suggestions within 3 seconds.
- 3.1.4 The system SHALL load and display a user profile within 2 seconds.

3.2 NFR-F2: Reliability

- 3.2.1 The system SHALL ensure that all posts, feedback, and deliverable submissions are saved without data loss.
- 3.2.2 The system SHALL send FYP deadline reminder emails within 5 minutes of the scheduled trigger time.

3.3 NFR-F3: Security

- 3.3.1 The system SHALL enforce role-based access control so that each user (Student, Team Lead, Advisor, Admin, FYP Office Staff) can only access features permitted to their role.
- 3.3.2 The system SHALL automatically log out a user after a period of inactivity.
- 3.3.3 The system SHALL store all user passwords in an encrypted format.
- 3.3.4 The system SHALL restrict access to team chats, project progress, and deliverable submissions so that only members of the respective team can view that data.

3.4 NFR-F4: Usability

- 3.4.1 The system SHALL allow a new user to create a profile, post an idea, and form a team without requiring any external guidance.
- 3.4.2 The system SHALL present conflict resolution tools (voting, dispute) in a clear and straightforward manner so that all team members can participate without confusion.

3.5 NFR-F5: Availability

- 3.5.1 The system SHALL maintain 99% uptime throughout the academic semester so that students can access team chats, posts, and project tracking at all times.

3.6 NFR-F6: Maintainability

- 3.6.1 The system SHALL be built with a modular structure so that features such as progress tracking, deliverable submission, and feedback can be updated or extended independently.

4. Analysis Class Diagram (ACD)

